Boosting Efficient Reinforcement Learning for Vision-and-Language Navigation With Open-Sourced LLM

Jiawei Wang, Teng Wang, Wenzhe Cai, Lele Xu, and Changyin Sun, *Senior Member, IEEE*

***Abstract*—Vision-and-Language Navigation (VLN) requires an agent to navigate in photo-realistic environments based on language instructions. Existing methods typically employ imitation learning to train agents. However, approaches based on recurrent neural networks suffer from poor generalization, while transformer-based methods are too large in scale for practical deployment. In contrast, reinforcement learning (RL) agents can overcome dataset limitations and learn navigation policies that adapt to environment changes. However, without expert trajectories for supervision, agents struggle to learn effective long-term navigation policies from sparse environment rewards. Instruction decomposition enables agents to learn value estimation faster, making agents more efficient in learning VLN tasks. We propose the Decomposing Instructions with Large Language Models for Vision-and-Language Navigation (DILLM-VLN) method, which decomposes complex navigation instructions into simple, interpretable sub-instructions using a lightweight, open-sourced LLM and trains RL agents to complete these sub-instructions sequentially. Based on these interpretable sub-instructions, we introduce the cascaded multi-scale attention (CMA) and a novel multi-modal fusion discriminator (MFD). CMA integrates instruction features at different scales to provide precise textual guidance. MFD combines scene, object, and action information to comprehensively assess the completion of sub-instructions. Experiment results show that DILLM-VLN significantly improves baseline performance, demonstrating its potential for practical applications.**

***Index Terms*—Vision-and-language navigation (VLN), large language models, reinforcement learning (RL), attention, discriminator.**

I. Introduction

VISION-AND-LANGUAGE navigation (VLN) task [1] requires an agent to navigate using visual information based on given language instructions, locating objects or scenes that match the instructions. VLN has gained significant research attention in recent years due to its vast potential in automation fields such as smart home assistants [2], [3]. Existing methods in the field of VLN mainly rely on imitation learning (IL), where agents learn to navigate based on language instructions by mimicking expert demonstrations. Initial VLN methods [1], [4], [5], [6] use recurrent neural networks as foundation models. However, these methods often exhibit poor generalization due to dataset biases, leading to significant performance drops in new or unseen environments. To overcome this limitation, researchers explore various data augmentation strategies to build more comprehensive datasets [7], [8], [9], [10]. However, creating a completely unbiased dataset is nearly impossible, and collecting expert trajectories is extremely costly. Recent VLN approaches have shifted to transformer-based models [11], [12], [13], [14], [15], which significantly improve navigation performance through large-scale pre-training on additional datasets. However, these models typically involve large model sizes, posing challenges for practical deployment.

Unlike IL, reinforcement learning (RL) overcomes dataset limitations by enabling agents to learn navigation policies autonomously through interactions with the environment. However, in VLN tasks, agents must handle multi-modal input and perform complex long-term sequential reasoning. Without expert trajectories for supervision, it is challenging for RL agents to learn effective navigation policies from sparse environment rewards. Humans, when navigating based on instructions, complete each part of the instruction step by step until reaching the destination. Inspired by this behavior, we aim to break down navigation instructions into simple sub-instructions. This approach allows agents to learn value estimation faster and more easily compared to predicting long-term value, as values are bootstrapped within the steps needed to complete each sub-instruction. This approach enables agents to learn and complete VLN tasks more efficiently. DISH [16] uses hierarchical RL to decompose navigation instructions into multiple latent intrinsic subgoals. However, these subgoals lack interpretability. Considering that interpretability is crucial for developing practical robotic applications, we aim to decompose instructions into simple and interpretable sub-instructions.

Large Language Models (LLMs) demonstrate exceptional semantic understanding and contextual awareness, showing immense potential in text analysis and comprehension. They provide powerful intelligent text processing support across various

Received 6 August 2024; accepted 18 November 2024. Date of publication 4 December 2024; date of current version 11 December 2024. This article was recommended for publication by Associate Editor J. Zhu and Editor M. Vincze upon evaluation of the reviewers' comments. This work was supported in part by the National Natural Science Foundation of China under Grant 62273093, Grant 62236002, and Grant 61921004, and in part by the "Zhishan" Scholars Programs of Southeast University. *(Corresponding author: Teng Wang.)*

Jiawei Wang is with the College of Electronics and Information Engineering, Tongji University, Shanghai 201804, China (e-mail: wangjw@tongji.edu.cn).

Teng Wang, Wenzhe Cai, and Lele Xu are with the School of Automation, Southeast University, Nanjing 210096, China (e-mail: wangteng@seu.edu.cn; wz_cai@seu.edu.cn; xulele@seu.edu.cn).

Changyin Sun is with the School of Automation, Southeast University, Nanjing 210096, China, and also with the School of Artificial Intelligence, Anhui University, Hefei 230601, China (e-mail: cysun@seu.edu.cn).

Digital Object Identifier 10.1109/LRA.2024.3511402

industries [17]. LLMs have been used to solve VLN tasks [18], [19], [20]. These methods typically convert visual observations into text prompts and employ close-sourced LLMs, like GPT-4, to determine the agent's actions. However, this visual-to-text conversion introduces noise, and these methods often rely on numerous prompts. This dependence on high-cost LLMs greatly limits their scalability in practical robotics. In this letter, we propose the DILLM-VLN (**D**ecomposing **I**nstructions with **L**arge **L**anguage **M**odels for **V**ision-and-**L**anguage **N**avigation) method. This method uses a lightweight, open-sourced LLM to decompose complex navigation instructions into multiple short, interpretable sub-instructions through simple prompts (Fig. 1). We then train RL agents to complete each sub-instruction sequentially. Based on these decomposed sub-instructions, we further design a Cascaded Multi-scale Attention (CMA) module and a Multi-modal Fusion Discriminator (MFD). During navigation, local attention effectively captures key features in the sub-instructions, while global attention extracts contextual information related to these key features. By integrating instruction features at different scales, the CMA provides accurate text features for the agent. The MFD integrates information about the agent's current scene, surrounding objects, and actions, allowing a comprehensive assessment of whether the agent successfully completes each sub-instruction.

Our contributions are summarized as follows: (1) We propose the DILLM-VLN method, which leverages LLM to break down complex navigation instructions into simple, interpretable sub-instructions, and RL agents are trained to complete each sub-instruction sequentially. Our method maintains competitive generalization performances without requiring ground truth actions for supervision while reducing model size significantly; (2) Based on the decomposed sub-instructions, we propose the cascaded multi-scale attention mechanism that extracts and integrates instruction features at different scales, providing the agent with accurate textual features; (3) Building on the interpretable sub-instructions, we design a novel multi-modal fusion discriminator that combines information from multiple modalities to comprehensively assess sub-instruction completion, enabling the agent to correctly switch to the next sub-instruction. Code is available at https://github.com/wangjw55/DILLM.

Fig. 1. LLM is used to decompose complex navigation instructions into multiple short sub-instructions. Each sub-instruction corresponds to a part of the navigation trajectory, making it easier for the agent to learn and complete these sub-instructions.

II. Related Work

A. Vision-and-Language Navigation (VLN)

Initial VLN methods [1], [4], [5], [6] rely on recurrent neural networks for agent navigation. However, these methods are limited by small-scale datasets, leading to overfitting and generalization challenges [21]. To address these issues, researchers propose various data augmentation methods. The speaker-follower frameworks [4], [5] use a speaker model to generate navigation instructions. Methods like EnvDrop [7], REM [8], and ENVEDIT [9] create new environments by modifying or combining existing ones. Lily [10] constructs a path-instruction dataset from YouTube house tour videos. Although these methods enhance model generalization by increasing data diversity, they cannot eliminate the bias between the data and the real world. An alternate adversarial learning method [22] is proposed, combining the strengths of teacher-forcing and student-forcing to improve navigation performance. Recently, transformer-based models [11], [12], [13], [14], [15], benefiting from powerful long-range dependency encoding, achieve significant performance gains through large-scale pre-training on additional datasets. However, these models often involve a vast number of parameters, which hinders practical deployment. RL enables agents to adapt to unknown and dynamic environments through continuous interaction with the environment, making it widely applied in agent navigation [23], [24]. RL is also used to solve VLN tasks [16], [25]. RPA [25] learns an environment model to simulate the real world for predicting future visual perceptions. However, the real world's state transitions are highly complex, and the environment model learned by RPA contains substantial noise, leading to poor navigation performance by the agents. DISH [16] employs a hierarchical reinforcement learning framework to decompose complex instructions into a series of intrinsic subgoals. However, this method suffers from poor interpretability, limiting its practical application. We propose using LLM to decompose instructions into sub-instructions and then train RL agents to complete these sub-instructions. Our trained agents not only demonstrate better generalization in unseen environments but also provide high interpretability, allowing us to understand the process of completing the entire navigation instruction.

B. Large Language Models (LLMs) Based VLN

LLMs [26], [27], [28] demonstrate impressive capabilities due to their extensive real-world knowledge and powerful reasoning abilities. In the field of embodied intelligence, LLM-based agents [29], [30] have garnered significant attention for solving complex tasks and show great potential, particularly in VLN tasks [18], [19], [20]. NavGPT [18] leverages the reasoning capabilities of LLMs to parse instructions, summarize observations, and integrate historical information, enabling zero-shot learning on the Room-to-Room (R2R) task [1]. MapGPT [19] enhances map-guided exploration by converting topological maps built online into prompts. However, these methods often rely on a large number of prompts, posing significant challenges to the model's input token limit and restricting their deployment in real-world robots. NaVid [31] encodes videos into visual

tokens, which are combined with instruction tokens and fed into the LLM for reasoning and action prediction. Recent studies [32], [33] show that integrating LLMs improves sample efficiency in RL tasks, enabling better generalization to new tasks or environment changes. We propose DILLM-VLN, which utilizes a lightweight, open-sourced LLM to decompose instructions and trains RL agents to complete the sub-instructions. This approach leverages the reasoning capabilities of LLM while reducing computational costs and redundancy, facilitating deployment in real-world robots.

III. Method

A. Problem Formulation

In the VLN task, the agent navigates based on the instruction $I = \{x_1, x_2, \ldots, x_L\}$, where L represents the instruction length and x_i is an individual word token. At any time step t, the agent receives a panoramic view P_t, divided into 36 separate views, each represented by an RGB image $\{p_{t,i}\}_{i=1}^{36}$. Due to impassable areas like walls, the agent can access only K navigable viewpoints $\{g_{t,k}\}_{k=1}^{K}$. Additionally, the agent obtains orientation information $(\theta_{t,i}, \phi_{t,i})$ for each view $p_{t,i}$, where $\theta_{t,i}$ and $\phi_{t,i}$ represent the heading and elevation angles, respectively.

All models involved in transforming image and text inputs to action outputs constitute the agent's navigation policy π_ω, where ω represents all parameters updated by RL algorithms. At each time step t, the agent predicts and executes an action a_t based on the navigation policy. The agent receives a reward r_t according to changes in its distance from the destination. Specifically, the agent receives a +1 reward if it gets closer to the destination and a -1 penalty if it moves farther away. This reward structure aims to encourage the agent to reach the destination quickly, thus improving navigation efficiency. When the agent performs the "stop" action, it is considered a successful navigation if the agent is within 3 meters of the destination, earning a +2 reward. Otherwise, it is deemed a navigation failure, incurring a -2 penalty. The goal of RL training is to find the optimal policy π_* that maximizes the expected discounted cumulative reward at any time t

$$\max_{\omega} \mathbb{E}_{\pi_\omega} \left[\sum_{k=0}^{T-1} \gamma^k r_{t+k} \right] \tag{1}$$

where T is the time step at which navigation ends, and γ is the discount factor. In the complex environment of the VLN task, we adopt a simple reward function design, as the navigation task is divided into multiple simple subtasks. This design effectively supports the agent in learning to complete these subtasks. The RL agent learns by maximizing cumulative rewards. Additional reward signals may introduce unnecessary distractions, leading the agent to pursue secondary objectives that deviate from the navigation goal, hindering its ability to learn efficient navigation policies. For experiment results on different reward function designs, please refer to our project repository.

B. Overall Framework of DILLM-VLN

Fig. 2 illustrates the overall framework of our proposed DILLM-VLN method. We use the pre-trained CLIP image encoder [34] to extract image feature $v_{t,i}$ for each view $p_{t,i}$. Orientation information $[\sin\theta_{t,i}, \cos\theta_{t,i}, \sin\phi_{t,i}, \cos\phi_{t,i}]$ is repeated 32 times to generate the orientation feature $o_{t,i}$. By concatenating the image feature and orientation feature, we obtain the visual feature $f_{t,i} = [v_{t,i}; o_{t,i}]$. For the natural language instruction I, we first use LLM to decompose the instruction into M sub-instructions $\{I^m\}_{m=1}^{M}$. We then tokenize and embed the instruction to obtain text embeddings $\{w_1, w_2, \ldots, w_L\}$. These text embeddings are encoded using a bidirectional LSTM to extract instruction features $\{U_1, U_2, \ldots, U_L\}$. Each sub-instruction I^m has corresponding instruction features $\{u_1^m, u_2^m, \ldots, u_{L^m}^m\}$, where L^m is the length of the sub-instruction I^m.

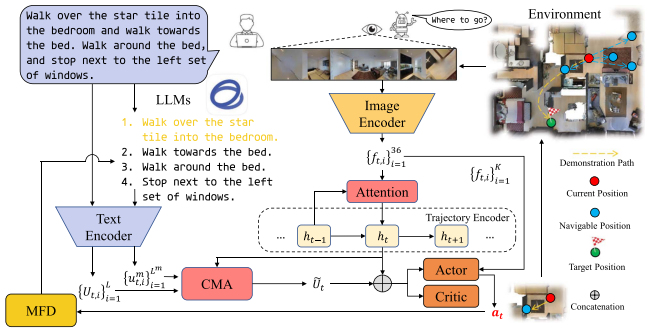

Fig. 2. Main architecture of DILLM-VLN.

We use LSTM as a trajectory encoder to encode the agent's historical trajectory and store its navigation history. At each time step t, the agent assigns attention weights to the visual features $f_{t,i}$ of each view based on the previous trajectory hidden state h_{t-1}. The agent then obtains the attentive visual feature $\tilde{f}_t$ through a weighted sum

$$a_{t,i} = \text{softmax}_\text{i} \left(f_{t,i}^T W_F h_{t-1} \right) \tag{2}$$

$$\tilde{f}_t = \sum_i a_{t,i} f_{t,i} \tag{3}$$

where W_F are learnable parameters. The trajectory encoder encodes $\tilde{f}_t$ and the previous trajectory hidden state h_{t-1} to obtain the current trajectory hidden state h_t

$$h_t = \text{LSTM} \left(\left[\tilde{f}_t; h_{t-1} \right] \right). \tag{4}$$

The CMA module infers key information and relevant context from instruction features and sub-instruction features based on h_t, generating the final text feature $\tilde{U}_t$. The actor module concatenates h_t and $\tilde{U}_t$ and calculates the similarity with each navigable viewpoint $g_{t,k}$. The probability $p_t(k)$ of the agent moving to the k-th navigable viewpoint is obtained by applying softmax to the similarity scores across all navigable points

$$p_t(k) = \text{softmax}_\text{k} \left(g_{t,k}^T W_G \left[\tilde{U}_t; h_t \right] \right) \tag{5}$$

where W_G are learnable parameters. After the agent moves to the next viewpoint, it employs the MFD to determine if the current sub-instruction is completed. If completed, the agent proceeds to execute the next sub-instruction; otherwise, it continues with the current one.

Following previous research [7], [16], we use the A2C algorithm to train the agent's navigation policy. As an on-policy RL algorithm, A2C converges smoothly and rapidly, which is particularly beneficial for training agents in complex VLN tasks. Accurately estimating long-term value is challenging, and since the agent completes each sub-instruction in a few steps, we set γ to 0 upon completion of each sub-instruction. This setting leverages the ability of LLM to decompose instructions into sub-instructions, truncating the temporal correlation between adjacent sub-instructions. As a result, the agent learns value estimation faster since the values bootstrap over fewer steps. To ensure accurate value estimation by the agent, we concatenate the text feature $\tilde{U}_t$ with the trajectory hidden state h_t as the state input for value estimation through the critic module.

C. Instruction Decomposition Based on LLM

We leverage the advantages of LLMs in text understanding and reasoning to break down complex long navigation instructions into concise short sub-instructions. We use the open-sourced ChatGLM-6B [27] model for instruction decomposition. Compared to other LLMs such as GPT [26] and LLaMA [28], ChatGLM-6B is lightweight, enabling users to deploy it locally on consumer-grade graphics cards. This not only enhances security but also facilitates the practical application of our method in real-world robots. We use the following prompt template to decompose each instruction:

Break the following instruction into multiple detailed sub-instructions and list them numerically, each sub-instruction must be written in English and end with a period, do not output other information: <instruction>

This prompt allow us to easily parse sub-instructions from the output. Since all instructions in the R2R dataset are in English, we specifically require the model to respond in English. To avoid hallucinations where LLM generate additional information, we explicitly forbid any output beyond instruction decomposition.

When the agent receives a new navigation instruction, it first uses LLM to break the instruction into a set of sub-instructions $\{I^m\}_{m=1}^{M}$. The agent then executes each sub-instruction sequentially. MFD determines if the agent has completed the current sub-instruction. If the discriminator confirms task completion, the agent proceeds to the next sub-instruction; otherwise, it continues with the current one.

Compared to traditional text segmentation method, which segment based on English grammar features at conjunctions and punctuation marks, LLM offers deeper text analysis capabilities and provides more reasonable segmentation results. Fig. 3 provides comparative examples of LLM and the traditional method in instruction decomposition.

The results from the decomposition of Instruction 1 show that the LLM understands "right" as a verb, generating the sub-instruction "turn right". However, the traditional method's straightforward decomposition leads to semantic ambiguity in the sub-instruction. In the R2R dataset, "right" is more often a modifier than an action, so the agent may fail to correctly interpret "right" as a command to turn right. For Instruction 2, the LLM understands that "turn right" should be completed "at the top" and includes both parts in the same sub-instruction. In contrast, the traditional method separates these parts, resulting in incomplete information for the agent. The decomposition of Instruction 3 indicates that the LLM recognizes the instruction as a description of a picture and does not decompose it, while the traditional method provides an incorrect decomposition. These findings demonstrate that the traditional method only splits instructions according to the original word order, failing to grasp semantic information. The LLM, with its semantic understanding capabilities, offers more reasonable segmentation results.

```
Instruction 1: Walk through doorway straight ahead, right immediately.
LLM:
["Walk through doorway straight ahead.", "Turn right immediately."]
Traditional Text Segmentation:
["Walk through doorway straight ahead", "right immediately"]

Instruction 2: Continue to climb the stairs and at the top, turn right and exit the room.
LLM:
["Continue to climb the stairs.", "At the top, turn right.", "Exit the room."]
Traditional Text Segmentation:
["Continue to climb the stairs", "and at the top", "turn right", "and exit the room"]

Instruction 3: Wait next to the painting that has a man and a watermelon in it.
LLM:
["Wait next to the painting that has a man and a watermelon in it."]
Traditional Text Segmentation:
["Wait next to the painting that has a man", "and a watermelon in it"]
```

Fig. 3. Comparative examples of LLM and the traditional method in instruction decomposition.

D. Cascaded Multi-Scale Attention (CMA)

In VLN task, navigation instructions are usually lengthy, and not all information is useful at every time step. Existing methods use attention mechanisms to dynamically adjust the focus on different parts of the instructions, adapting to the navigation process. However, this requires the agent to accurately estimate its current progress, or it risks mismatching the focused instructions with the actual position. Our proposed method decomposes the instructions, allowing the agent to access only the relevant parts needed for the current navigation step. By applying attention to the sub-instruction being executed, the agent can focus on critical objects or directional information, facilitating the selection of correct navigation actions. Despite this, due to the coherence of navigation instructions, a single sub-instruction might not contain all necessary information, requiring the agent to retrieve some details from the context.

For example, when executing the instruction *"Go through the next hallway that is on the left diagonal to you, follow it around, and stop at the French doors."*, the agent cannot understand "it" refers to the "hallway" if provided only with the sub-instruction "follow it around". The lack of context leads to navigation failure. To address this issue, we propose the cascaded multi-scale attention. CMA uses the information extracted by local attention to guide global attention, and then integrates feature information from different scales to guide the agent's action selection, thus improving navigation accuracy. The framework of CMA is shown in Fig. 4.

At the local level, we apply an attention mechanism to sub-instructions, enabling the agent to focus on critical objects or directional information. Specifically, at time step t, the agent

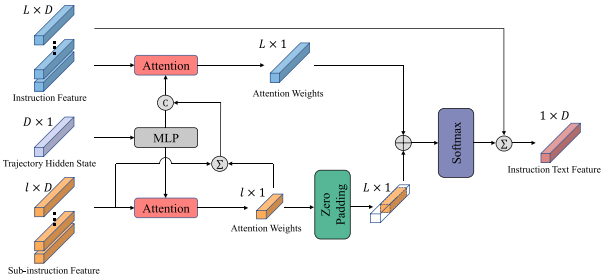

Fig. 4. Overall framework of CMA. "C" denotes concatenation, "$\sum$" represents weighted sum, and "+" indicates element-wise sum.

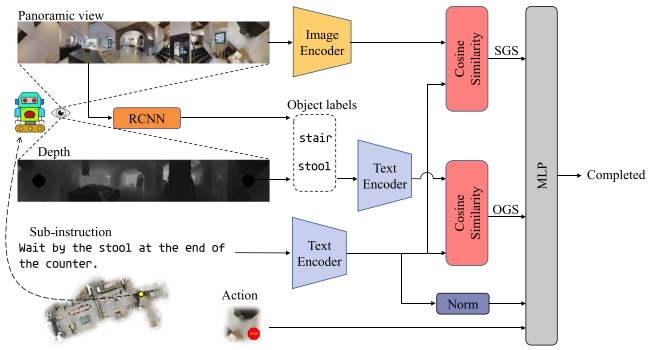

Fig. 5. Overall framework of MFD.

is executing sub-instruction I^m. Based on the trajectory hidden state h_t, attention weights are assigned to each word feature of the sub-instruction, and the local attentive sub-instruction feature $\tilde{f}_t^m$ is obtained through weighted sum

$$\beta_{t,i}^m = \text{softmax}_i \left(\left(u_{t,i}^m \right)^T W_u h_t \right) \tag{6}$$

$$\tilde{f}_t^m = \sum_i \beta_{t,i}^m u_{t,i}^m \tag{7}$$

where W_u are learnable parameters. At the global level, we use local attentive sub-instruction feature $\tilde{f}_t^m$ and h_t to guide the application of the attention mechanism to the entire instruction. This enables the agent to extract contextual information relevant to local key information. Specifically, the agent calculates the attention weight for each instruction word based on h_t and $\tilde{f}_t^m$

$$\mu_{t,i} = \text{softmax}_i \left(U_{t,i}^T W_U \left[h_t; \tilde{f}_t^m \right] \right) \tag{8}$$

where W_U are learnable parameters. The number of sub-instruction attention weights is L^m. For words not included in the sub-instructions, their attention weights are set to zero, ensuring the total number of sub-instruction attention weights remains L. We obtain the final attention weights by performing an element-wise sum of the local attention weights and the local-guided global attention weights, followed by applying a softmax function. Finally, we derive the instruction text feature U_t through a weighted sum

$$\rho_{t,i} = \text{softmax}_i \left(\beta_{t,i}^m + \mu_{t,i} \right) \tag{9}$$

$$\tilde{U}_t = \sum_i \rho_{t,i} U_{t,i} \tag{10}$$

E. Multi-Modal Fusion Discriminator (MFD)

Since we decompose the instructions into multiple sub-instructions and execute them sequentially, it is necessary to create a discriminator to assess whether the current sub-instruction is complete. This can be translated into the problem of matching the agent's current state with the sub-instruction. CLIPNav [35] uses the CLIP model to achieve this by matching the agent's current panoramic view with the extracted keyphrase. However, this approach has limitations. In the VLN task, the criterion for completing an instruction is the agent stopping within a certain threshold distance from the target location. The agent's view has no distance limit, so the agent might see the object mentioned in the instruction and assume it has succeeded, even though it is still outside the target distance threshold. Additionally, sub-instructions generated by the LLM include simple commands like"turn left", which cannot be judged as complete by matching the sub-instruction with the agent's view. To address these issues, we propose a multi-modal fusion discriminator that combines scene, object, and action information to comprehensively determine whether the current sub-instruction is completed. The framework of MFD is shown in Fig. 5.

First, we use the CLIP image encoder to extract scene features from the agent's view and the CLIP text encoder to extract text features from sub-instructions. By calculating the cosine similarity between each scene feature and sub-instruction text feature, we obtain the scene grounding score (SGS), indicating whether the agent has reached the scene described by the sub-instruction (e.g.,"hallway", "bathroom"). Next, we use Fast-RCNN [36] pre-trained on the Visual Genome dataset to extract objects from each individual view and obtain the depth information of the object center pixels using the Matterport3D simulator. We retain only the object labels within the threshold distance of the agent (3 meters in R2R) and use the CLIP text encoder to extract features of these object labels. We then calculate the cosine similarity between these features and the sub-instruction text features to obtain the object grounding score (OGS), indicating whether the agent has found the target object in the sub-instruction (e.g.,"guitar","table"). Finally, we combine the agent's actions with the sub-instruction text features to determine whether the agent has completed the action described in the sub-instruction. Specifically, at time step t, we concatenate SGS_t, OGS_t, the agent's action a_t, and the normalized instruction feature U_t^{CLIP}, and input them into a trainable MLP to predict whether the agent has completed the current sub-instruction

$$\text{done}_t = \text{MLP} \left(SGS_t, OGS_t, a_t, U_t^{\text{CLIP}} \right). \tag{11}$$

IV. Experiment

A. Experiment Setup

Experiment Data: We validate the proposed algorithm on the R2R dataset (21567 instructions), which is divided into a training set (14025 instructions), a seen validation set (1020 instructions), an unseen validation set (2349 instructions), and

TABLE I
RESULTS OF DIFFERENT VLN METHODS ON THE R2R DATASET

	Methods	Params	Validation Seen				Validation Unseen				Test Unseen			
			NL↓	NE↓	SR↑	SPL↑	NL↓	NE↓	SR↑	SPL↑	NL↓	NE↓	SR↑	SPL↑
Imitation Learning Methods														
1	seq2seq [1]	13.25M	11.3	6.01	38.6	-	8.39	7.81	21.8	-	**8.13**	7.85	20.4	0.18
2	RPA [25]	-	**8.46**	5.56	42.9	-	**7.22**	7.65	24.6	-	9.15	7.53	25.3	0.23
3	Speaker-Follower [4]*	25.49M	-	3.36	66.4	-	-	6.62	35.5	-	14.8	6.62	35.0	0.28
4	RCM+SIL(train) [5]*	-	10.6	3.53	66.7	-	11.5	6.09	42.8	-	12.0	6.12	43.0	0.38
5	Self-Monitoring [6]*	25.23M	-	3.22	67	0.58	-	5.52	45	0.32	18.0	5.99	43	0.32
6	RecBERT [12]†	159.99M	9.78	3.92	62	0.59	10.3	5.10	50	0.46	11.1	5.45	51	0.47
7	HOP [13]†	158.39M	10.7	3.50	66	0.63	11.8	4.74	54	0.49	12.5	4.93	55	0.50
8	HAMT [11]†	170.83M	11.15	**2.51**	**76**	**0.72**	11.46	**2.29**	66	**0.61**	12.27	3.93	65	**0.60**
9	DUET [14]†	239.18M	-	-	-	-	13.94	3.31	**72**	0.60	14.73	**3.65**	**69**	0.59
Zero-shot Methods														
10	NavGPT [18]	-	-	-	-	-	11.45	6.46	34	0.29	-	-	-	-
11	MapGPT [19]	-	-	-	-	-	-	6.29	38.8	0.26	-	-	-	-
12	DiscussNav [20]	-	-	-	-	-	**9.69**	**5.32**	**43**	**0.40**	-	-	-	-
Reinforcement Learning Methods														
13	A2C	13.25M	30.7	6.08	35.2	0.12	31.1	6.58	32.0	0.11	30.5	6.65	27.3	0.09
14	PPO	13.25M	25.3	5.16	45.2	0.27	24.4	6.01	37.7	0.22	25.4	6.05	38.0	0.24
15	DISH [16]	27.43M	**12.1**	**4.36**	**60.9**	**0.54**	**10.2**	6.05	44.7	0.40	**10.5**	6.01	44.3	0.41
16	DILLM-VLN	12.07M	12.8	4.74	57.2	0.51	11.4	**5.31**	**49.4**	**0.44**	11.6	**6.00**	**46.3**	**0.43**

* indicates back translation augmentation.† indicates additional dataset. Black denotes the best results of each method category. ↑ means higher is better, while ↓ means smaller is better.

a test set (4173 instructions). The environments in the seen validation set appear in the training set. The environments in the unseen validation set and the test set do not appear in the training set, allowing us to assess the model's generalization ability.

Evaluation Metrics: The evaluation metrics include: Navigation Length (NL): The average length of the agent's navigation trajectory in meters. Navigation Error (NE): The average distance between the agent's final position and the target position in meters. Success Rate (SR): The proportion of times the agent ends up within 3 meters of the target position. Success Rate Weighted by Path Length (SPL): Balances navigation success rate and trajectory length. We use SR as the key metric and select the optimal training model based on SR.

Implementation Details We use a pre-trained CLIP-RN50x4 model to extract image features. All instructions are pre-decomposed into sub-instructions using LLMs to accelerate training. The model is trained for approximately four days on two 2080Ti GPUs, totaling 100K steps. If the LLM is called via API, only one 2080Ti GPU is required. The maximum episode length is set to 15 steps. We use the AdamW optimizer for parameter updates, with a batch size of 64 and a learning rate of 1e-4. For more detailed parameter settings, refer to our code repository.

B. Experiment Results

Table I shows the performance of our proposed DILLM-VLN method and other methods on the R2R dataset. Rows #1 to #9 list IL-based methods, rows #10 to #12 display zero-shot methods, and rows #13 to #16 show RL-based methods. Among all RL-based methods, DILLM-VLN achieves the highest SR on the test unseen set, improving by 69.6% and 12.8% over the A2C and PPO baselines, respectively. Additionally, DILLM-VLN requires the fewest training parameters. Unlike DISH, which uses a hierarchical structure for additional subgoal prediction training, our method employs LLM to decompose instructions, reducing the training parameters by half and increasing SR by 4.5%,

TABLE II
RESULTS OF ABLATION STUDY

	Methods	Validation Seen				Validation Unseen			
		NL↓	NE↓	SR↑	SPL↑	NL↓	NE↓	SR↑	SPL↑
1	DILLM-VLN	12.8	**4.74**	**57.2**	**0.51**	11.4	**5.31**	**49.4**	**0.44**
2	DILLM-VLN w/o LLM	13.1	5.63	53.2	0.48	10.9	5.84	47.9	0.43
3	DILLM-VLN w/o CMA	**11.6**	5.62	51.5	0.47	11.2	6.25	44.8	0.41
4	DILLM-VLN w/o MFD	12.1	5.27	54.8	0.50	**10.5**	5.79	47.4	0.43

w/o means without.

highlighting DILLM-VLN's parameter efficiency. Compared to GPT-based zero-shot methods, DILLM-VLN achieves better performance on the val unseen set through low-cost in-domain training, even with a lighter LLM. Compared to DiscussNav, DILLM-VLN improves SR by 14.9%. Compared to IL-based methods, DILLM-VLN performs better than RNN-based models (rows #1 to #5) even without using ground truth actions as supervision signals. Although there is still a gap between DILLM-VLN and the latest transformer-based models (rows #6 to #9) in terms of SR, DILLM-VLN is more lightweight and easier to deploy in practical robotics applications. Moreover, the LLM-based decomposition in DILLM-VLN offers greater interpretability, helping us understand how agents progressively complete navigation tasks, which is crucial for developing practical robotic applications.

C. Ablation Study

We investigate the contribution of each proposed module by removing them individually from the DILLM-VLN model. The results are presented in Table II. First, we replace the LLM-based instruction decomposition with the traditional text segmentation method, splitting instructions at conjunctions, commas, and periods. Comparing rows #1 and #2, using the traditional segmentation method increases NE by 10.0% and decreases SR by 3.0% on the validation unseen set. This occurs because the traditional segmentation method lacks the text comprehension ability of LLM, leading to inaccurate sub-instructions and poor navigation performance. Next, we remove the CMA module,

Fig. 6. Navigation sample of the proposed DILLM-VLN method. The dark or light color on the instruction indicates the relative attention value of CMA (the darker the larger). The red arrow indicates the selected action, 'stop' indicates the end of the navigation.

TABLE III
RESULTS OF TESTING DILLM-VLN WITH DIFFERENT LLMS

	LLMs	Validation Seen				Validation Unseen			
		NL↓	NE↓	SR↑	SPL↑	NL↓	NE↓	SR↑	SPL↑
1	ChatGLM-6B	12.8	**4.74**	**57.2**	**0.51**	**11.4**	**5.31**	**49.4**	**0.44**
2	GPT-3.5	12.8	5.03	55.6	0.49	11.8	5.34	48.8	0.43
3	LLaMA-70B	**11.9**	5.04	55.4	0.50	11.5	5.48	48.8	**0.44**

and the agent allocates attention weights to the current sub-instruction word features based solely on the trajectory hidden state h_t. Comparing rows #1 and #3, using the simple attention mechanism increases NE by 17.7% and decreases SR by 9.3% on the validation unseen set. The simple attention mechanism fails to capture necessary contextual information, resulting in inaccurate navigation. This demonstrates that CMA is crucial for DILLM-VLN, helping the agent in acquiring essential contextual information and improving navigation SR. Finally, we remove the MFD module. To ensure the agent can still switch sub-instructions properly, we let the agent switch to the next sub-instruction every fixed three steps. Comparing rows #1 and #4, using a fixed-step strategy increases NE by 9.0% and decreases SR by 4.0% on the validation unseen set. The fixed-step strategy does not meet the actual navigation needs because the steps required for each sub-instruction vary. This confirms the importance of the discriminator in judging sub-instruction completion, allowing the agent to switch sub-instructions flexibly based on completion. Through these experiments, we confirm the effectiveness of using LLM for instruction decomposition and the CMA and MFD modules designed based on the decomposed sub-instructions. These modules are compatible and jointly improve the agent's navigation performance.

In recent years, various LLMs with different parameter scales have emerged. To evaluate the impact of different LLMs on the performance of the DILLM-VLN method, we conducted comparative experiments using GPT-3.5 and LLaMA-70B. The experimental results are shown in Table III. The results indicate that changing LLMs has little impact on DILLM-VLN performance. While differences in parameter scale and training methods can lead to performance variations, our DILLM-VLN does not heavily rely on high-performance LLMs. It only requires the ability to decompose instructions into reasonable sub-instructions, a capability that all tested LLMs possess. The sub-instructions decomposed by different LLMs are essentially the same. Therefore, we selected the more lightweight ChatGLM-6B. Unlike other LLMs that typically require API access, ChatGLM-6B is easily deployable locally, offering greater security and convenience.

D. Qualitative Visualization

To intuitively demonstrate the function of our designed modules, we visualize a complete navigation process of the agent in Fig. 6. First, the LLM sequentially decomposes the instruction into simple sub-instructions, laying the foundation for the agent to complete the navigation task. Next, we visualize the agent's attention distribution to the instruction and sub-instructions when using the CMA module. By observing the changes in the agent's attention during navigation, we find that local attention enables the agent to focus on key actions, directions, or object information within the current sub-instruction. Meanwhile, locally guided global attention allows the agent to obtain relevant contextual information based on the current sub-instruction. For example, when the agent executes the sub-instruction "go down the hall" in steps 3 and 4, local attention captures the key information "hall", while locally guided global attention makes the agent notice "bedroom", helping it determine that reaching the "bedroom" is necessary to complete the current sub-instruction. Finally, we output MFD's prediction on whether the previous sub-instruction has been completed at each step. The results indicate that MFD accurately determines whether the agent has completed the sub-instruction, enabling the selection of the appropriate next sub-instruction for execution.

V. CONCLUSION

In this letter, we propose DILLM-VLN to address the VLN task. DILLM-VLN uses the LLM to decompose complex navigation instructions into multiple simple and interpretable

sub-instructions and trains an RL agent to execute each sub-instruction sequentially. This approach does not rely on ground truth actions as supervision. Based on these interpretable sub-instructions, we propose the CMA mechanism and a novel MFD. CMA extracts key information from sub-instructions through local attention while using this local information to guide global attention, extracting related contextual information to provide accurate textual features for the agent. MFD combines scene, object, and action information to comprehensively assess the completion of sub-instructions, allowing the agent to correctly switch to the next sub-instruction. Extensive experiment results demonstrate that our proposed DILLM-VLN method outperforms existing baseline methods. Specifically, our model achieves a 46.3% SR and 0.43 SPL on unseen test sets. We believe that DILLM-VLN offers a simple and attractive prototype for combining LLMs and RL to solve complex robotic tasks. In future work, we plan to extend this method to address more complex VLN tasks in continuous action spaces.

REFERENCES

[1] P. Anderson et al., "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 3674–3683.

[2] L. Yue, D. Zhou, L. Xie, F. Zhang, Y. Yan, and E. Yin, "Safe-VLN: Collision avoidance for vision-and-language navigation of autonomous robots operating in continuous environments," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 4918–4925, Jun. 2024.

[3] A. Magassouba, K. Sugiura, and H. Kawai, "Crossmap transformer: A crossmodal masked path transformer using double back-translation for vision-and-language navigation," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 6258–6265, Oct. 2021.

[4] D. Fried et al., "Speaker-follower models for vision-and-language navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 3318–3329.

[5] X. Wang et al., "Reinforced cross-modal matching and self-supervised imitation learning for vision-language navigation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6629–6638.

[6] C.-Y. Ma et al., "Self-monitoring navigation agent via auxiliary progress estimation," 2019, *arXiv:1901.03035*.

[7] H. Tan et al., "Learning to navigate unseen environments: Back translation with environmental dropout," in *Proc. NAACL HLT - Conf. N. Am. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol. - Proc. Conf.*, 2019, pp. 2610–2621.

[8] C. Liu, F. Zhu, X. Chang, X. Liang, Z. Ge, and Y.-D. Shen, "Vision-language navigation with random environmental mixup," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 1644–1654.

[9] J. Li, H. Tan, and M. Bansal, "EnvEdit: Environment editing for vision-and-language navigation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15407–15417.

[10] K. Lin, P. Chen, D. Huang, T. H. Li, M. Tan, and C. Gan, "Learning vision-and-language navigation from Youtube videos," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 8317–8326.

[11] S. Chen, P.-L. Guhur, C. Schmid, and I. Laptev, "History aware multimodal transformer for vision-and-language navigation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5834–5847.

[12] Y. Hong, Q. Wu, Y. Qi, C. Rodriguez-Opazo, and S. Gould, "VLN BERT: A recurrent vision-and-language bert for navigation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1643–1653.

[13] Y. Qiao, Y. Qi, Y. Hong, Z. Yu, P. Wang, and Q. Wu, "HOP: History-and-order aware pre-training for vision-and-language navigation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 15418–15427.

[14] S. Chen, P. -L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, "Think global, act local: Dual-scale graph transformer for vision-and-language navigation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16537–16547.

[15] L. Wang, Z. He, R. Dang, M. Shen, C. Liu, and Q. Chen, "Vision-and-language navigation via causal learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 13139–13150.

[16] J. Wang, T. Wang, L. Xu, Z. He, and C. Sun, "Discovering intrinsic subgoals for vision-and-language navigation via hierarchical reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, May 15, 2024, doi: 10.1109/TNNLS.2024.3398300.

[17] C. Tang, D. Huang, W. Ge, W. Liu, and H. Zhang, "GraspGPT: Leveraging semantic knowledge from a large language model for task-oriented grasping," *IEEE Robot. Autom. Lett.*, vol. 8, no. 11, pp. 7551–7558, Nov. 2023.

[18] G. Zhou, Y. Hong, and Q. Wu, "NavGPT: Explicit reasoning in vision-and-language navigation with large language models," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 7641–7649.

[19] J. Chen et al., "MapGPT: Map-guided prompting for unified vision-and-language navigation," in *Proc. Annu. Meet. Assoc. Comput. Linguist.*, 2024, pp. 9796–9810.

[20] Y. Long, X. Li, W. Cai, and H. Dong, "Discuss before moving: Visual language navigation via multi-expert discussions," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 17380–17387.

[21] Y. Zhang, H. Tan, and M. Bansal, "Diagnosing the environment bias in vision-and-language navigation," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 890–897.

[22] W. Zhang, C. Ma, Q. Wu, and X. Yang, "Language-guided navigation via cross-modal grounding and alternate adversarial learning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 9, pp. 3469–3481, Sep. 2021.

[23] Y. Zhu et al., "Target-driven visual navigation in indoor scenes using deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 3357–3364.

[24] J. Wang, T. Wang, Z. He, W. Cai, and C. Sun, "Towards better generalization in quadrotor landing using deep reinforcement learning," *Appl. Intell.*, vol. 53, no. 6, pp. 6195–6213, 2023.

[25] X. Wang, W. Xiong, H. Wang, and W. Y. Wang, "Look before you leap: Bridging model-free and model-based reinforcement learning for planned-ahead vision-and-language navigation," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 37–53.

[26] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.

[27] Z. Du et al., "GLM: General language model pretraining with autoregressive blank infilling," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 320–335.

[28] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[29] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "Voxposer: Composable 3D value maps for robotic manipulation with language models," in *Proc. Conf. Robot Learn*, 2023, pp. 540–562.

[30] Z. Yang et al., "Text2reaction: Enabling reactive task planning using large language models," *IEEE Robot. Autom. Lett.*, vol. 9, no. 5, pp. 4003–4010, May 2024.

[31] J. Zhang et al., "Navid: Video-based VLM plans the next step for vision-and-language navigation," 2024, *arXiv:2402.15852*.

[32] T. Carta et al., "Grounding large language models in interactive environments with online reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 3676–3713.

[33] B. Wang et al., "LLM-empowered state representation for reinforcement learning," 2024, *arXiv:2407.13237*.

[34] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.

[35] V. S. Dorbala et al., "Clip-NAV: Using clip for zero-shot vision-and-language navigation," 2022, *arXiv:2211.16649*.

[36] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 91–99.